School ERP — Deployment Guide (Docker)

Deploy on AWS EC2 or Oracle Cloud · one image, one container per school · click any Copy button to grab the command.

Legend: BOTH do on AWS & Oracle AWS AWS only ORACLE Oracle only

What you need: a server (AWS EC2 Ubuntu 22.04+ with ≥2 GB RAM, Oracle Ampere A1 up to 24 GB — Always Free, or any Ubuntu VPS), a domain where you can add DNS records (e.g. `school1.mydomain.in`), and your key file. You deploy the **same image** once per school — only the env file, port and domain differ.

Already deployed and just updating? Go straight to **Step 17**. **Adding a second or third school** to a server that's already running one? Everything you need is in **Step 15** — Steps 1–8 are done once per server, not per school.

1 Create the server + a STATIC IP BOTH

- AWS Launch an EC2 **Ubuntu** instance (t3.small/medium). Allocate an **Elastic IP** and associate it. Security Group inbound: **22, 80, 443**.
- ORACLE Create an **Ampere A1 Ubuntu** instance. Give it a **Reserved Public IP** (Oracle's Elastic IP). Add VCN Security List ingress for **80** and **443**.

Use a **static** IP (Elastic / Reserved). A default ephemeral IP changes if the box stops/starts and breaks your domain.

2 Connect over SSH BOTH

Replace the key file and IP. Note the flag is a lowercase `-i`.

```
ssh -i your-key.pem ubuntu@YOUR_STATIC_IP
```

3 Add swap ORACLE AWS

Recommended on 1 GB boxes. **Skip on the 24 GB Oracle A1.**

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
free -h
```

4 Open the host firewall ORACLE

Oracle Ubuntu images ship a restrictive firewall. AWS users **skip this** — the Security Group already handles it.

```
sudo ufw allow 22
sudo ufw allow 80
sudo ufw allow 443
sudo ufw --force enable
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 80 -j ACCEPT
sudo iptables -I INPUT 6 -m state --state NEW -p tcp --dport 443 -j ACCEPT
sudo netfilter-persistent save
```

5 Update the system BOTH

```
sudo apt-get update && sudo apt-get upgrade -y
```

6 Install Docker + Compose BOTH

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
```

Now **log out and back in** (close SSH and reconnect) so `docker` works without `sudo`. Then verify:

```
docker version
docker compose version
```

Common error — "permission denied ... /var/run/docker.sock". If a later `docker` or `docker compose` command fails with `permission denied while trying to connect to the docker API at unix:///var/run/docker.sock`, your current session hasn't picked up the `docker` group yet. Fix it without logging out by running this, then re-run your command:

```
newgrp docker
```

(Logging out and back in also fixes it.) Confirm Docker now works **without sudo**:

```
docker ps
```

7 Install host Nginx + Certbot + Git BOTH

Nginx (on the host) handles the domain routing + free HTTPS in front of the containers.

```
sudo apt-get install -y nginx certbot python3-certbot-nginx git
```

8 Clone the project BOTH

```
sudo mkdir -p /opt/sfms && sudo chown $USER:$USER /opt/sfms
cd /opt/sfms
git clone https://github.com/meprashantkumar/school-mangement-system-new.git app
cd app
```

9 Configure the first school BOTH

Copy the template and fill in this school's values.

```
cp .env.docker.example school1.env
nano school1.env
```

Set at least: `WEB_PORT=8081`, `CLIENT_URL=https://school1.mydomain.in`, a strong `MONGO_PASS`, a **unique** `JWT_SECRET`, `SCHOOL_NAME`, and the `SUPERADMIN_*` login. Generate a JWT secret with:

```
openssl rand -hex 32
```

Save + exit nano: **Ctrl+O**, Enter, then **Ctrl+X**. This file holds secrets — it's gitignored, never commit it.

School branding — all 16 variables. The same env file carries the whole public identity. Fill these with *this* school's details; anything left blank falls back to a built-in default (which is *R K Public School*, so a blank line on another school shows the wrong name):

```
VITE_SCHOOL_NAME            # "Sunrise Academy" — shown in headers everywhere
VITE_SCHOOL_PLACE           # "Ranchi" — the subtitle under the name. Blank = "Garhwa"!
VITE_SCHOOL_FULL_NAME       # "Sunrise Academy, Ranchi" — receipts, report cards
VITE_SCHOOL_SHORT_NAME      # "SAR" — tight spaces
VITE_SCHOOL_MONOGRAM        # "SA" — the crest badge (1-3 letters)
VITE_SCHOOL_TAGLINE         # one line under the name on the home page
VITE_SCHOOL_INTRO           # the paragraph on the home page
VITE_SCHOOL_DIRECTOR_NAME   # "Mr. A N Panday"
VITE_SCHOOL_DIRECTOR_ROLE   # "Director"  (blank = Director)
VITE_SCHOOL_PRINCIPAL_NAME  # "Mr. S Panday" — also signs report cards
VITE_SCHOOL_PRINCIPAL_ROLE  # "Principal" (blank = Principal)
VITE_SCHOOL_ADDRESS         # full postal address — footer + receipts
VITE_SCHOOL_PHONE           # office phone
VITE_SCHOOL_EMAIL           # office email
VITE_SCHOOL_ESTABLISHED     # "1998"
VITE_SCHOOL_AFFILIATION     # "CBSE" / "JAC"
```

These are baked into the site **at build time**, so they apply when you run the `--build` in Step 10. To change them later, edit the env file and re-run with `--build` — a plain restart will **not** pick them up. Editing `frontend/.env` on the server does nothing at all: that file is excluded from the Docker build context.

`SCHOOL_NAME` and `VITE_SCHOOL_NAME` are two different settings — set both, to the same value. `VITE_SCHOOL_NAME` is the frontend (what parents see in the app). `SCHOOL_NAME` is the backend: it prints on **receipts**, goes out in **emails**, and now **names every backup file**. A printed receipt takes its heading from the backend and its address line from the frontend build, so if the two disagree the receipt contradicts itself.

Money & fee policy — check these before go-live.

- `RAZORPAY_KEY_ID` / `RAZORPAY_KEY_SECRET` — this school's own Razorpay account, so online fees land in its bank. Leave blank to disable online payment (the counter QR still works).
- `SCHOOL_UPI_VPA` — this school's UPI id for the counter QR. Parents scanning it pay the school directly with no gateway cut.
- `ONLINE_PLATFORM_FEE_PCT` — convenience fee added to online payments, as a **percentage**. Default `2.5`, which covers Razorpay's 2% + 18% GST. Note the `_PCT` suffix: a line named `ONLINE_PLATFORM_FEE` is silently ignored.
- `LATE_FEE_PER_DAY` / `LATE_FEE_MAX` — automatic late fee per day past the due date, and its cap. Both default to `0` (feature off).

- `DB_NAME` — give each school its own (see Step 15; sharing one name makes a wrong-school restore possible).

Two things that crash the app / block login if you skip them:

- **SUPERADMIN_PASSWORD must be at least 6 characters.** A shorter one makes the API crash on first boot and login shows a **502**. (The template's `admin` is only 5 — change it.)
- Use **real values** for `SUPERADMIN_EMAIL` / `SUPERADMIN_PASSWORD` — not the `admin@school1.example.com` placeholder. Pick an email you own (needed for password resets).
- `WEB_PORT` must match the port in your Nginx block (Step 12).

10 Build & start the school BOTH

The `-p` project name isolates this school's containers and database volume.

```
docker compose --env-file school1.env -p sfms_school1 up -d --build
```

Check it (use this school's WEB_PORT):

```
docker compose -p sfms_school1 ps
curl http://localhost:8081/api/health
```

You should see `{"status":"ok","service":"sfms-api"}`.

Always include `--env-file school1.env`. Without it none of your settings load — the port falls back to 8080 and the database starts with no password, so the API crash-loops. If login shows **502**, or `docker ps` shows `api … Restarting`, read the exact reason:

```
docker compose -p sfms_school1 logs api --tail=30
```

Most common causes: a super-admin password under 6 characters, or a Mongo auth mismatch — fix the value in `school1.env`, then re-run the `up` command above.

11 Point the domain BOTH

In your DNS provider add an **A record**: `school1.mydomain.in → YOUR_STATIC_IP`. Wait until it resolves:

```
ping school1.mydomain.in
```

12 Host Nginx reverse proxy BOTH

```
sudo nano /etc/nginx/sites-available/schools
```

Paste this (one `server` block per school — port must match WEB_PORT):

```
server {
    listen 80;
    server_name school1.mydomain.in;
    client_max_body_size 2g;
    location / {
        proxy_pass http://127.0.0.1:8081;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 600s;
        proxy_send_timeout 600s;
    }
}
```

Enable it and reload:

```
sudo ln -sf /etc/nginx/sites-available/schools /etc/nginx/sites-enabled/schools
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

13 Get free HTTPS (Certbot) BOTH

Choose **redirect HTTP → HTTPS** when asked. Certbot also auto-renews.

```
sudo certbot --nginx -d school1.mydomain.in
```

14 Verify BOTH

```
curl -I https://school1.mydomain.in
curl https://school1.mydomain.in/api/health
```

Open the site in a browser, log in with your `SUPERADMIN_EMAIL` / `SUPERADMIN_PASSWORD`, then **change the super-admin password in the app**.

15 Add another school (2nd, 3rd, 4th…) on its own domain BOTH

One server runs as many schools as its RAM allows. Each school is a **complete, isolated stack** — its own MongoDB container, its own data volume, its own API and web container. Nothing is shared except the host Nginx and the Docker engine, so one school can never see another's data. **Repeat this one section per school**; nothing in Steps 1–8 is needed again.

A. Plan the three values that make it separate

	School 1	School 2	School 3
Env file	`school1.env`	`school2.env`	`school3.env`
Project name (`-p`)	`sfms_school1`	`sfms_school2`	`sfms_school3`
WEB_PORT	`8081`	`8082`	`8083`
DB_NAME	`rkps`	`sunrise`	`stmary`
Domain	school1.mydomain.in	sunrise.mydomain.in	stmary.mydomain.in

The `-p` project name is what isolates everything: it prefixes the containers *and* the data volume, so school 2's database lives in `sfms_school2_mongo_data` and school 1's in `sfms_school1_mongo_data`. The domains can also be completely different registered domains — nothing here assumes a shared parent domain.

B. What MUST differ between the env files

Setting	Why it must be its own
`WEB_PORT`	**Hard failure if shared** — two containers can't bind one host port, so the second school won't start.
`RAZORPAY_KEY_ID` `RAZORPAY_KEY_SECRET`	**Money.** Shared keys send both schools' online fees into one bank account.
`SCHOOL_UPI_VPA`	**Money.** The counter QR must pay the school the parent is standing in.
`DB_NAME`	See the warning below — this one is silent and unrecoverable.
`CLIENT_URL`	This school's own `https://` domain. Used for CORS and for links inside emails.
`SCHOOL_NAME` + all 16 `VITE_SCHOOL_*`	The branding itself. `SCHOOL_NAME` also names this school's backup files.
`SUPERADMIN_EMAIL` `SUPERADMIN_PASSWORD`	Each school's office gets its own administrator account.
`JWT_SECRET` , `MONGO_PASS`	Hygiene: one school's leaked secret must not open the other.
`BACKUP_REMOTE`	A separate Drive folder per school, e.g. `gdrive:sfms-backups-school2` .

Fine to keep identical: `MONGO_USER` , `JWT_EXPIRE` , `ONLINE_PLATFORM_FEE_PCT` , `LATE_FEE_PER_DAY` , `LATE_FEE_MAX` , `MONGO_CACHE_GB` , and the `EMAIL_*` block if one mailbox sends for both.

⚠ **Give every school its own `DB_NAME` — before any data is entered.** Day to day, two schools both using `sfms` is harmless: separate Mongo containers, separate volumes, no mixing. The danger is at **restore** time. A `mongodump` archive carries the name of the database it came from and will only load into a database of that same name — so if both are called `sfms` , **school 1's backup file restores cleanly into school 2**. No error, no mismatch, no warning; you have simply overwritten the wrong school. Distinct names turn that mistake into a harmless refusal.

Changing `DB_NAME` *after* a school has records does not move anything — the API just connects to a new empty database and the data looks like it vanished (it's still there under the old name, merely unused). Renaming needs a dump and reload, and **the order matters**: copy the data out *while the env file still points at the old name*. Renaming `sfms` → `sunrise` :

```
# 1. Dump FIRST — school2.env still says DB_NAME=sfms at this point
cd /opt/sfms/app
```

```
docker compose --env-file school2.env -p sfms_school2 exec -T api sh -c \
  'mongodump --uri="$MONGO_URI" --gzip --archive' > /tmp/old.archive.gz

# 2. Reload it under the new name (--nsTo does the renaming)
docker compose --env-file school2.env -p sfms_school2 exec -T api sh -c \
  'mongorestore --uri="$MONGO_URI" --gzip --archive --nsFrom="sfms.*" --nsTo="sunrise.*"' <
/tmp/old.archive.gz

# 3. NOW point the app at the new name
nano school2.env                                          # DB_NAME=sunrise
docker compose --env-file school2.env -p sfms_school2 up -d

# 4. Check the data is there, then drop the old database
docker compose -p sfms_school2 exec -T api sh -c \
  'mongosh "$MONGO_URI" --quiet --eval "db.students.countDocuments()"'
```

Do **not** drop the old `sfms` database until step 4 shows the expected student count. If the count is 0, the app is looking at an empty database — re-check `DB_NAME` and that step 2 actually ran.

C. Create and start the new school

```
cd /opt/sfms/app
cp .env.docker.example school2.env
nano school2.env          # set WEB_PORT=8082, DB_NAME, domain, branding, keys

docker compose --env-file school2.env -p sfms_school2 up -d --build
```

Confirm it's healthy on **its own** port before touching Nginx:

```
docker compose -p sfms_school2 ps
curl http://localhost:8082/api/health
```

You want `{"status":"ok","service":"sfms-api"}` and the api container `Up`, not `Restarting`. School 1 keeps running throughout — you never stop it to add another school.

D. Point the new domain at the same server

In your DNS provider add an **A record** for the new domain to the **same static IP** — every school on this box shares one IP, and Nginx tells them apart by `server_name`.

```
sunrise.mydomain.in.   A   YOUR_STATIC_IP
```

```
ping sunrise.mydomain.in
```

Wait until it answers with your server's IP. Certbot in step F **will fail** if DNS hasn't propagated yet.

E. Add a second Nginx server block

Open the same file you made in Step 12 and **append** another block — don't replace the first one:

```
sudo nano /etc/nginx/sites-available/schools
```

```
server {
    listen 80;
    server_name sunrise.mydomain.in;        # the NEW domain
    client_max_body_size 2g;
```

```
    location / {
        proxy_pass http://127.0.0.1:8082;   # the NEW school's WEB_PORT
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 600s;
        proxy_send_timeout 600s;
    }
}
```

Only two lines change per school: `server_name` and the `proxy_pass` port. Test the config and reload:

```
sudo nginx -t && sudo systemctl reload nginx
```

> `reload` is graceful — school 1 stays up and no parent is disconnected. Never use `restart` here.

F. HTTPS for the new domain

Certbot only touches the domain you name, so school 1's certificate is left alone. Choose **redirect HTTP → HTTPS** when asked.

```
sudo certbot --nginx -d sunrise.mydomain.in
```

```
curl -I https://sunrise.mydomain.in
curl https://sunrise.mydomain.in/api/health
```

Open it in a browser: you should see the **new school's** name and crest, not school 1's. If you see school 1's branding, the `VITE_SCHOOL_*` lines were blank or missing when you built — fill them in and re-run the `up -d --build`.

G. Backups for the new school

rclone does not need connecting again. The config lives on the host at `./rclone/rclone.conf` and every school's container mounts that same file, so once you've done Step 16 on this box, all schools can already reach Drive. Just give the new school its own folder and its own cron line:

```
docker compose -p sfms_school2 exec api rclone lsd gdrive:
docker compose -p sfms_school2 exec api rclone mkdir gdrive:sfms-backups-school2
```

```
sudo crontab -e
```

```
15 2 * * * cd /opt/sfms/app && docker compose --env-file school2.env -p sfms_school2 exec -T api sh -c 'mongodump --uri="$MONGO_URI" --gzip --archive | rclone rcat "$BACKUP_REMOTE/school2-$(date +\%F).archive.gz"' >> /var/log/sfms-backup-school2.log 2>&1
```

> **Stagger the times** — 2:00 for school 1, 2:15 for school 2, 2:30 for school 3. Two `mongodump`s at once on a small box fight for CPU and disk. The in-app **Back up to Google Drive** button also works per school, and its filenames now start with that school's `SCHOOL_NAME` so the two are never confused in Drive.

H. How many schools fit?

Each school adds one MongoDB, one API and one Nginx container. Mongo is the memory-hungry one, and `MONGO_CACHE_GB` (default **1**) caps its cache. Rough guide for schools of 500–1000 students:

Server RAM	Comfortable schools

2 GB (+ swap)	1
4 GB	2
8 GB	4–5
24 GB (Oracle A1)	10+

Disk and bandwidth are never the limit — a 1000-student school's database stays well under 1 GB a year.

Handy per-school commands — every one of them needs the right `-p`, which is how you pick the school:

```
docker compose -p sfms_school2 ps                    # status
docker compose -p sfms_school2 logs api --tail=50    # why is it unhappy
docker compose -p sfms_school2 restart api           # restart just this school
docker compose -p sfms_school2 down                  # stop it (data volume is KEPT)
docker volume ls | grep mongo_data                   # see each school's volume
```

`down` without `-v` never deletes data. **Never** add `-v` unless you intend to erase that school permanently.

16 Nightly backup to Google Drive BOTH

First-time setup — connect rclone to Google Drive (once per SERVER, not per school). This runs inside a school's api container but is saved on the host at `./rclone/rclone.conf`, and every school on the box mounts that same file — so do it once and all schools can reach Drive. Use `-it` so the prompts are interactive:

```
docker compose -p sfms_school1 exec -it api rclone config
```

Answer the prompts like this:

```
n                     # New remote
name> gdrive          # MUST be "gdrive" — it matches BACKUP_REMOTE in your env file
Storage> drive        # Google Drive
client_id>            # paste your own, or Enter to leave blank (see note)
client_secret>        # paste your own, or Enter to leave blank
scope> 1              # 1 = full access
service_account_file> # Enter (blank)
Edit advanced config? > n
Use auto config? > n    # ★ NO — the server is headless, it has no browser
```

Leaving `client_id` / `client_secret` blank uses rclone's shared key (works, just slower / rate-limited). For your own key, create an **OAuth client (Desktop app)** in Google Cloud Console → APIs & Services → Credentials.

rclone then prints a `rclone authorize "drive" "…"` line and waits at `config_token>`. **Leave the server terminal open** and run that **exact** line on your **Windows PC** (the blob carries your client id/secret). Install rclone there first:

```
winget install Rclone.Rclone
```

```
rclone authorize "drive" "<paste the exact blob the server printed>"
```

A browser opens → sign into the backup Google account → **Allow**. Copy the `{ … }` JSON it prints, paste it into the server's waiting `config_token>` prompt, then answer `n` (not a Shared Drive), `y` (confirm), `q` (quit).

403 — "Google Drive API has not been used in project … or it is disabled". If you used your **own** client id/secret, you must enable the Drive API on that Google Cloud project **once**: open `https://console.cloud.google.com/apis/api/drive.googleapis.com`, select the same project, click **Enable**, and wait ~2 minutes. Also answer **No** to *"Configure this as a Shared Drive (Team Drive)?"* — you're backing up to a normal Drive; answering Yes makes rclone list Team Drives, which triggers this same 403.

Backups stop after ~7 days? Publish the OAuth consent screen. If you used your own client id/secret and the project's OAuth consent screen is in **Testing** mode, Google expires the refresh token after 7 days and backups quietly start failing. Fix once: Google Cloud Console → **APIs & Services** → **OAuth consent screen** → **Publish app** (move to *In production*). For a single owner this works even without Google verification — just dismiss the "unverified app" notice.

Windows error — "bind: An attempt was made to access a socket in a way forbidden by its access permissions" on port 53682. Docker Desktop / WSL reserves that port, so rclone can't open its login callback. Fix: in an **Administrator** Command Prompt, temporarily stop Windows NAT, run the authorize, then start NAT again.

```
net stop winnat
rclone authorize "drive" "<paste the exact blob the server printed>"
net start winnat
```

`net stop winnat` needs an elevated prompt — Start → type **cmd** → right-click → **Run as administrator**. Restarting winnat restores Docker Desktop networking.

Verify the connection and create the school's backup folder:

```
docker compose -p sfms_school1 exec api rclone lsd gdrive:
docker compose -p sfms_school1 exec api rclone mkdir gdrive:sfms-backups-school1
```

Then schedule a 2 AM dump that streams straight to Drive. Edit the host crontab:

```
sudo crontab -e
```

Add one line per school (note the escaped `\%` — cron needs it):

```
0 2 * * * cd /opt/sfms/app && docker compose --env-file school1.env -p sfms_school1 exec -T api sh -c
'mongodump --uri="$MONGO_URI" --gzip --archive | rclone rcat "$BACKUP_REMOTE/school1-$(date
+\%F).archive.gz"' >> /var/log/sfms-backup-school1.log 2>&1
```

The in-app **Backup → "Back up to Google Drive now"** button also works once rclone is connected. Your data lives in the `mongo_data` volume — this backs it up **off the server**.

Change the Google (Gmail) account later? Do it **on the server** (not your laptop) — rclone runs the backups there, so changing it locally has no effect. Re-authorize the same remote to the new account:

```
# non-Docker box (SSH into the server first):
rclone config reconnect gdrive:

# Docker setup:
docker compose -p sfms_school1 exec api rclone config reconnect gdrive:
```

When it asks "*Use web browser to automatically authenticate?*" answer **No** (the server has no browser). It prints an `rclone authorize "drive" …` line — run that on a machine **with a browser** (your PC), sign in with the **new Gmail**, then paste the token back into the server prompt. Verify the account switched:

```
rclone about gdrive:
rclone lsd gdrive:
```

Old backups stay in the old account's Drive; new backups go to the new account from now on.

17 Update to the latest code BOTH

Pull the newest code from GitHub and rebuild the image. Your **database volume is untouched** — students, fees and payments stay put. Do this per school (one `up` line each).

New env lines your existing school files won't have yet. Two rounds of settings have been added since the first deployments. **Before** the rebuild below:

1. Run `git pull origin main` first — this refreshes `.env.docker.example`.
2. Copy any missing lines from it into each school's env file and fill in that school's real values: the 16 `VITE_SCHOOL_*` branding lines (Step 9), plus `SCHOOL_NAME`, `DB_NAME`, `LATE_FEE_PER_DAY`, `LATE_FEE_MAX`, `BACKUP_REMOTE` and `ONLINE_PLATFORM_FEE_PCT`.
3. Then run the `up -d --build` command below — the `--build` is what bakes the branding in.

Skip it and the site still works, but shows the default *R K Public School* branding and quietly runs with late fees off. A plain restart won't apply branding changes — it must be a `--build`. Careful with `DB_NAME` on a school that **already has data**: changing it points the API at a new empty database (see Step 15).

What the latest code adds. Nothing here needs configuration — it's listed so you know what changes for the office after the rebuild:

- **Mobile-number login for parents and teachers.** Most parents have no email, so the office now issues logins: **Students** → 🔑 on a row, or **Give class access** for a whole class at once (printable slips, or one shared password). Forgotten passwords are reset by the office, not by email. Existing email logins keep working.
- **Automatic one-time migration on first boot.** The new code rebuilds two indexes (email/phone become unique + sparse) and normalises stored mobile numbers into one form, so a number saved as `08235083251` still matches its children. It also gives every existing holiday a scope (see below) and replaces the holidays index. All of it is idempotent and needs no action — you'll just see it in `logs api`:

  ```
  [holidays] marked 1 existing holiday(s) whole-school
  [holidays] dropped the old dateKey-only unique index
  ```

  There is **never** a manual database step for an update — if a release needs a schema change, the api applies it when it boots.
- **Holidays: vacations and single classes. Attendance → Holidays this session** now takes a **date range**, so a summer vacation is one action instead of 45, and shows as one line with a single **Remove all**. Each holiday also has a scope: **Whole school** (the default, and what every existing holiday became) or **Specific classes**, for a board-exam or result day that closes Class 10 only. A class holiday leaves everyone else — and the **staff register** — working as normal; only a whole-school holiday closes those. Sundays inside a range are skipped as already-weekly-offs, a day that is already a holiday keeps its own name, and if attendance was already taken on days in the range the office is shown which days and asked to confirm (the records are kept either way). A class teacher can still close a **single** day for the school; ranges and class holidays are office-only.
- **Fee generation:** tick **which classes** and **which fees** to bill each month — so an exam fee in a month when only Class 2 sits an exam goes to Class 2 alone. Adds **Undo a Month**, and fixes a bug that could bill a student twice after a fee structure was edited.
- **Teacher and parent dashboards rebuilt for phones** — that's where they're actually used.
- **Online payment fix.** A parent logging in by mobile previously got a 403 when paying; that check also let a parent open receipts belonging to other children. Both closed.
- **Backup files are named after the school** (from `SCHOOL_NAME`) instead of a fixed `RKPS`, so two schools' archives can't be mixed up in Drive.

```
cd /opt/sfms/app
git pull origin main
docker compose --env-file school1.env -p sfms_school1 up -d --build
docker compose --env-file school2.env -p sfms_school2 up -d --build
```

Then confirm each school is healthy on its own WEB_PORT:

```
docker compose -p sfms_school1 ps
curl http://localhost:8081/api/health
```

You should see `{"status":"ok","service":"sfms-api"}` and the api container `Up` (not `Restarting`).

Backup button not working? You're on old code. The in-app **"Back up to Google Drive now" / "Download backup"** feature only exists in newer commits. If the button errors with `spawn mongodump ENOENT` or *"mongodb-database-tools not installed"*, that box simply hasn't been updated yet — run the `git pull` + rebuild above. The `mongodump` tool itself is baked **into the image** (installed by the Dockerfile), so once you're on the latest code the button works with no extra install.

If `git pull` complains about local changes: your `.env` files are gitignored, so they're never touched. To force the code to exactly match GitHub, run `git reset --hard origin/main` then re-run the rebuild. This only resets tracked **code** — it never deletes your env files or the database volume.

Old images pile up over time. Reclaim disk after a few updates with `docker image prune -f` (safe — only removes dangling images, never volumes).

18 Change a school's settings (edit its `.env`) BOTH

To change any setting — school name, super-admin email, UPI id, backup remote, etc. — edit that school's env file and **re-run the same `up` command**. Compose recreates the container with the new values. No `--build` is needed when only the env changed (the code is the same).

```
cd /opt/sfms/app
nano school1.env                                          # edit the values, Ctrl+O Enter, Ctrl+X
docker compose --env-file school1.env -p sfms_school1 up -d   # apply — recreates the container
docker compose -p sfms_school1 logs api --tail=20         # confirm it came back up
```

Combining a code update **and** an env change? Just add `--build` to the same command: `docker compose --env-file school1.env -p sfms_school1 up -d --build`.

Two env values need a matching change elsewhere:

- **Changed `WEB_PORT`?** Update this school's Nginx `proxy_pass` port (Step 12) to match, then `sudo nginx -t && sudo systemctl reload nginx` — otherwise you get a **502**.
- **Changed the domain / `CLIENT_URL`?** Update `server_name` in the Nginx block, reload Nginx, and re-run `sudo certbot --nginx -d newdomain` for a fresh certificate.

⚠ **Do NOT change `MONGO_USER` / `MONGO_PASS` after first boot.** Those only create the database user **once**, on the very first start when the `mongo_data` volume is still empty. Editing them later does **not** update the existing DB user, so the api can no longer authenticate and crash-loops with an auth error (login shows **502**). If you truly must rotate the DB password, change it inside MongoDB itself — or wipe the volume, which **erases all data** (take a backup first). To change the **super-admin** password after go-live, do it in the app (Profile → change password), not in the env file.

19 Make it future-proof BOTH

- Pin base images to a digest in the Dockerfiles (e.g. `node:20.20.2-bookworm-slim@sha256:…`) so they never drift.
- Save the built image with your backups, so it works even if a base image disappears:

```
docker save sfms_school1-api | gzip > sfms-api-image.tar.gz
```

Also allocate the **static IP** (Step 1), keep the repo on GitHub, and store the per-school `.env` files safely (they hold your secrets).

20 Several schools on ONE small box (1 GB) SMALL BOX

Steps 1–19 give every school its **own MongoDB** and put **host Nginx + Certbot** in front. That is the right shape when the box has RAM to spare — a database server per school is the strongest boundary you can draw.

It does not fit on 1 GB. One `mongod` wants roughly 250 MB for its cache alone, so three of them claim ~750 MB before the three Node APIs get a single byte. The box swaps constantly, or the OOM killer starts picking off containers.

So this track is a second way to run **the same images**, built for a small box:

- **One MongoDB** holding **one database per school** (`sfms_school1`, `sfms_school2`, ...) instead of one server each.
- **Caddy** instead of host Nginx + Certbot. It gets and renews each domain's HTTPS certificate by itself — no `certbot`, no renewal cron, no reload — and it gzips responses, which the container Nginx did not.

Schools stay separate: own database, own JWT secret, own containers, own branding. Only the database *server* is shared. Two files drive it, both in `deploy/multi-school/`: `docker-compose.infra.yml` (Caddy + Mongo, once per box) and `docker-compose.school.yml` (api + web, once per school).

A. Prepare the box

Ubuntu, ports **22, 80, 443** open in the AWS security group, a **static/Elastic IP**, and **swap** (Step 3) — on 1 GB, swap is what stops the frontend build being killed. Then Docker (Step 6). You do **not** need host Nginx or Certbot on this track, so Step 7 and Steps 11–13 are skipped.

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER      # log out and back in so docker works without sudo
git clone https://github.com/YOUR_USER/YOUR_REPO.git app
cd app/deploy/multi-school
```

B. DNS first

Point an **A record** for every domain at the box *before* starting Caddy. Caddy proves ownership over port 80, so a domain that does not resolve yet just fails and retries.

```
school1.mydomain.in   A   YOUR_STATIC_IP
school2.mydomain.in   A   YOUR_STATIC_IP
school3.mydomain.in   A   YOUR_STATIC_IP
```

C. Shared services (once per box)

```
cp infra.env.example infra.env
nano infra.env        # ACME_EMAIL, MONGO_USER, MONGO_PASS (openssl rand -hex 24)
chmod 600 infra.env

nano Caddyfile        # one site block per domain -> that school's web container

docker compose --env-file infra.env -f docker-compose.infra.yml -p sfms_infra up -d
```

Certificates live in the `caddy_data` volume. Keep it: losing it means re-issuing every certificate, and Let's Encrypt allows only 5 per domain per week.

D. Each school

```
cp school.env.example school1.env
nano school1.env
chmod 600 school1.env

docker compose --env-file school1.env -f docker-compose.school.yml -p sfms_school1 up -d --build
```

Repeat per school. Build them **one at a time** on a small box — two `vite build`s at once will exhaust the RAM.

Must differ per school: `SCHOOL_KEY`, `DB_NAME`, `JWT_SECRET`, `CLIENT_URL`. Give two schools the same `DB_NAME` and they silently share one set of students, with no error to warn you. Share a `JWT_SECRET` and a login for one school is accepted by the other.
Must be the same: `MONGO_USER` / `MONGO_PASS` — it is one database server, and those are its root credentials.

E. Verify

```
docker ps                                    # infra + 2 containers per school
curl https://school1.mydomain.in/api/health  # {"status":"ok","service":"sfms-api"}
curl -s https://school1.mydomain.in/ | grep -o '<title>[^<]*'   # this school's own name
docker exec school1-api sh -c 'echo $MONGO_URI'   # check it names THIS school's DB
free -m                                      # headroom left
```

Worth confirming isolation directly: log in on school 1, then send that token to school 2 — it must come back **401**, because the secrets differ.

F. Adding a 4th school later

```
# 1. DNS A record for school4.mydomain.in -> YOUR_STATIC_IP
# 2. add a block to the Caddyfile:
#      school4.mydomain.in {
#              import school school4-web
#      }
docker exec sfms-caddy caddy reload --config /etc/caddy/Caddyfile

# 3. its own env file, then bring it up
cp school.env.example school4.env && nano school4.env
docker compose --env-file school4.env -f docker-compose.school.yml -p sfms_school4 up -d --build
```

The Caddy reload is graceful: no dropped connections, and existing certificates are not re-issued. The other schools are untouched.

G. Updating to new code

```
cd ~/app && git pull
cd deploy/multi-school
for s in school1 school2 school3; do
  docker compose --env-file $s.env -f docker-compose.school.yml -p sfms_$s up -d --build
done
```

One at a time, and the databases are untouched. Remember branding is compiled in, so an edit to any `VITE_SCHOOL_*` value needs `--build`, not just a restart.

H. Why the two networks

Every school's compose file has services literally named `api` and `web`. Put them all on one shared Docker network and those names are ambiguous — one school's Nginx could resolve `api` to **another school's backend**. That is a data leak, and it would depend on Docker's DNS ordering rather than on anything you control.

So there are two networks, and which container joins which is the whole point:

- `sfms_edge` — Caddy + every `web`. Caddy asks for `school1-web`, which is unique.
- `sfms_dbnet` — Mongo + every `api`. Each api asks for `sfms-mongo`, which is unique.
- api ↔ web talk over the school's **own private** network, where `api` can only mean that school's api.

> The rule that keeps it true: no `web` is ever on `dbnet`, and no `api` is ever on `edge`.

I. Backups on this track

One shared Mongo means **one thing to back up and one thing to lose**. Restoring a single school means restoring one database out of the archive, not the whole volume — so dump **per school**, never just the volume, or a single school's restore will overwrite all of them.

```
docker exec school1-api sh -c 'mongodump --uri="$MONGO_URI" --gzip --archive' \
  > school1-$(date +%F).archive.gz
```

An archive carries the database name it came from, so restoring into a differently-named database needs `--nsFrom` / `--nsTo` (Step 16). Then off-box, one folder per school, as in Step 16.

J. How much fits in 1 GB

Measured with three schools live: about **520 MB** used, **~380 MB** free, swap barely touched. Roughly Mongo 250 MB (cache capped at `0.25`), each api ~70 MB (heap capped at `192`), each Nginx ~10 MB, Caddy ~20 MB.

- **3 schools** on 1 GB: comfortable for a few hundred students each.
- **4–5**: possible, but raise the swap and watch `docker stats`.
- **More than that**, or bigger schools: move to 2–4 GB and go back to Steps 1–19, one Mongo per school.

The caps matter in both directions. Node sizes its heap from total system RAM, so without `API_HEAP_MB` three APIs each assume they may use half the box; and the frontend build needs `BUILD_HEAP` raised to 1 GB or `tsc` gets OOM-killed part-way through.